

Exercise 1 Analysis: Comparison of RLS, (1+1)EA, and Genetic Algorithm

Introduction

This analysis compares the performance of three algorithms - RLS, (1+1)EA (OnePlusOneEA), and my Genetic Algorithm from Assignment 2 - across 11 different problem instances. Each algorithm was run 30 times on each instance with a budget of 10,000 function evaluations. The plots show the mean performance with standard deviation bands.

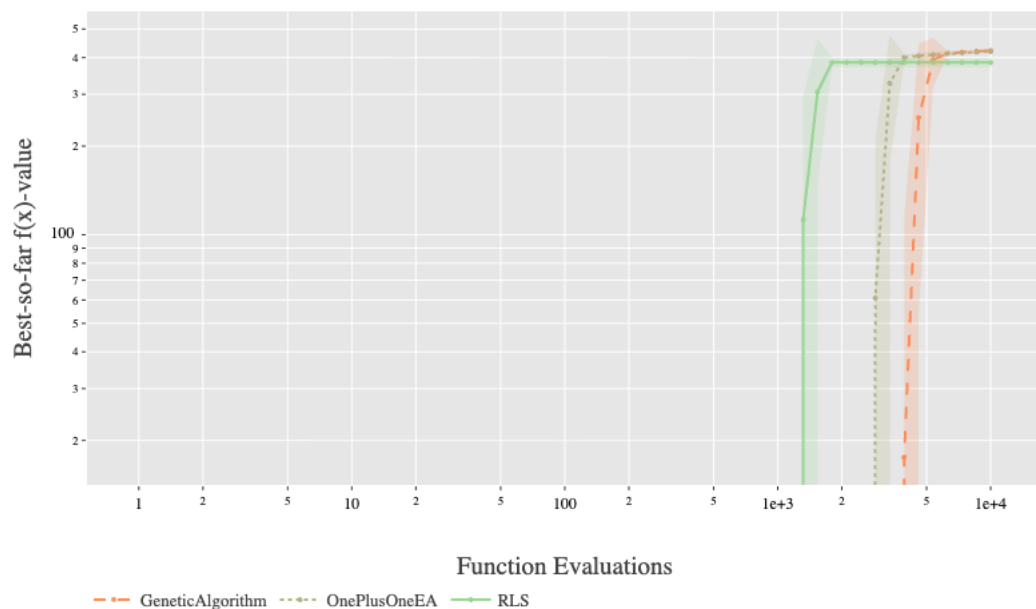
The instances include:

- MaxCoverage: 2100, 2101, 2102, 2103
- MaxInfluence: 2200, 2201, 2202, 2203
- PackWhileTravel: 2300, 2301, 2302

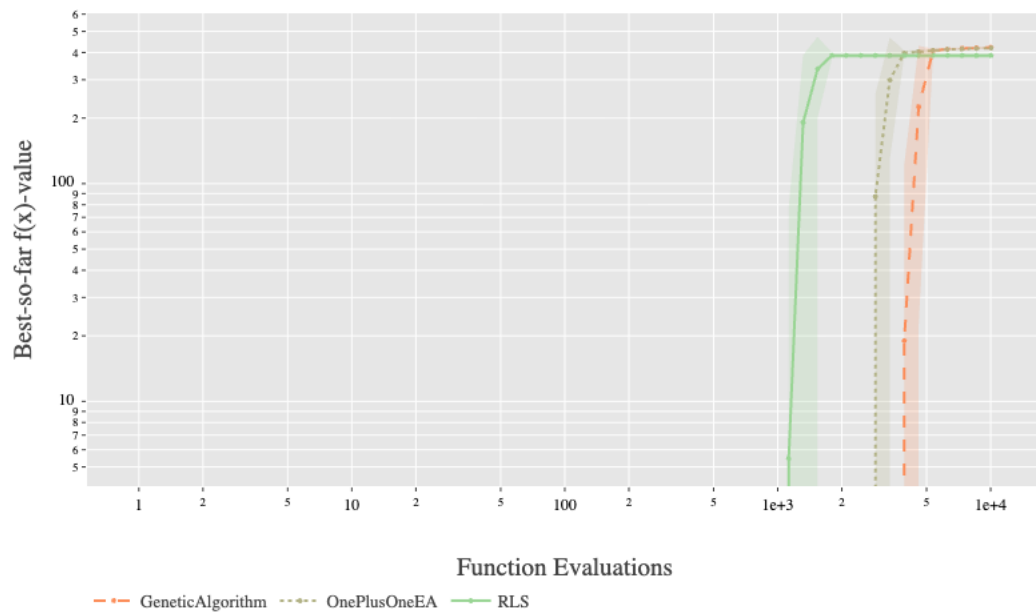
Results by Problem Type

MaxCoverage Problems (2100-2103)

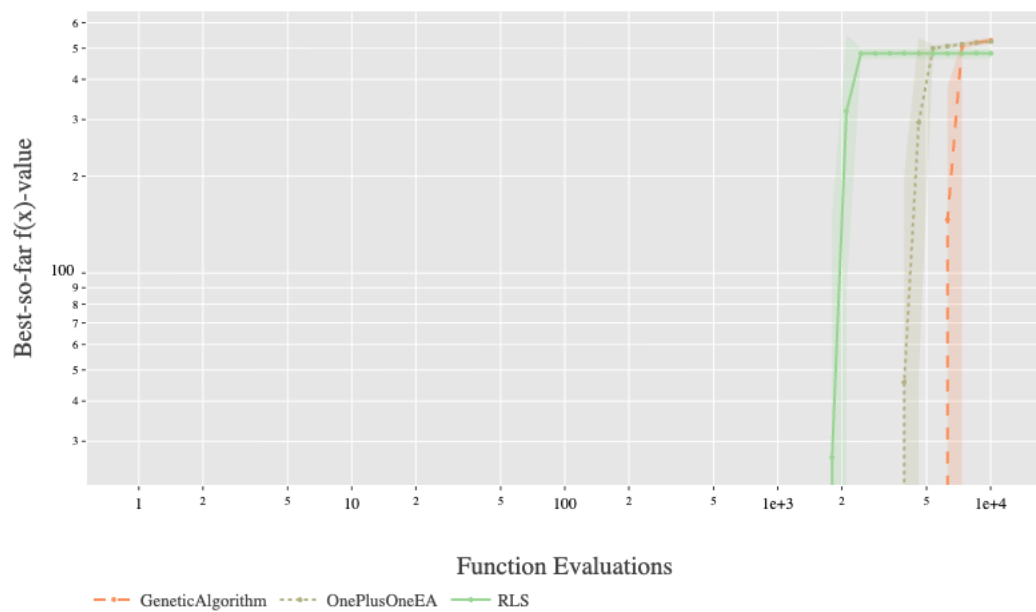
Looking at the four MaxCoverage instances, there's a consistent pattern across all of them:



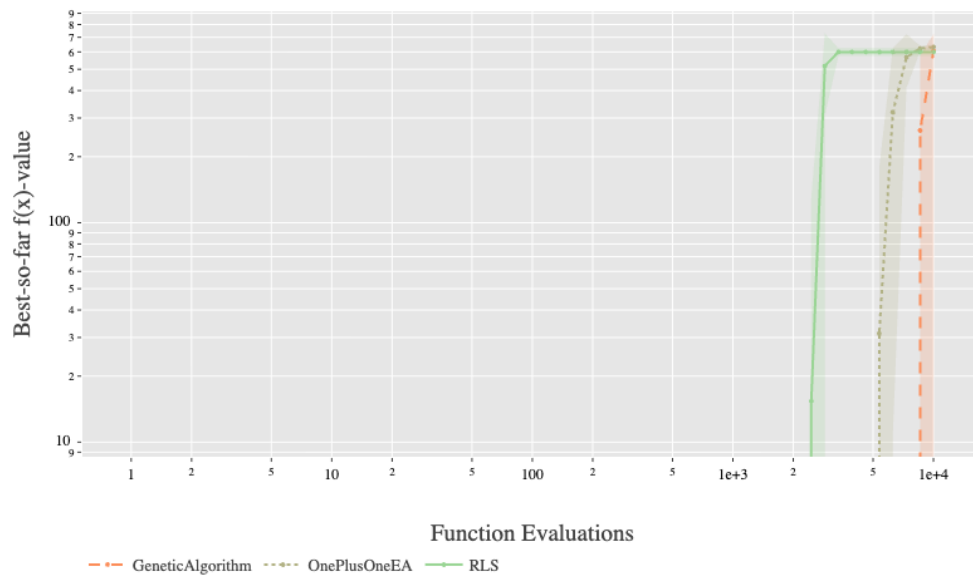
Instance 2100: RLS (green line) reaches around 4.0 fitness and converges fastest, getting there by about 1,500-2,000 evaluations. OnePlusOneEA (dotted line) performs very similarly to RLS, reaching the same fitness level but maybe slightly slower. My Genetic Algorithm (orange dashed line) reaches the same final fitness but takes more evaluations to get there - it's still improving even past 2,000 evaluations.



Instance 2101: Pretty much the same story as 2100. RLS converges quickest, OnePlusOneEA is close behind, and the GA is slower. All three reach around 4.0 fitness. The standard deviation bands look tighter here, meaning the runs were more consistent.



Instance 2102: Similar pattern but now the algorithms reach around 5.0 fitness. RLS still fastest, OnePlusOneEA similar, GA slower. The gap between the algorithms is a bit more visible on this instance.

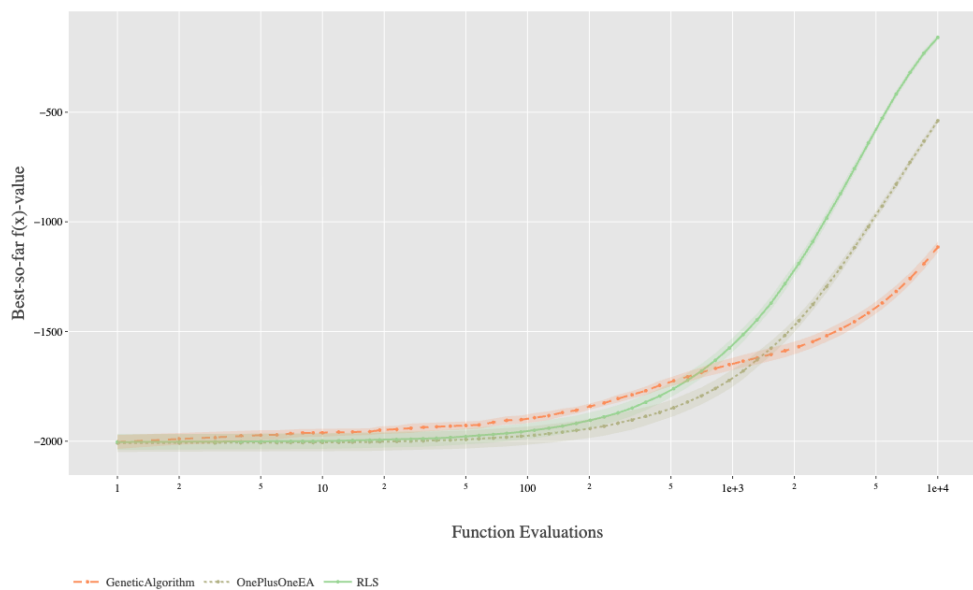


Instance 2103: All three algorithms reach about 6.0 fitness. RLS converges first (around 1,000-1,500 evals), then OnePlusOneEA, then the GA. By 10,000 evaluations they're all at roughly the same place, but RLS got there much faster.

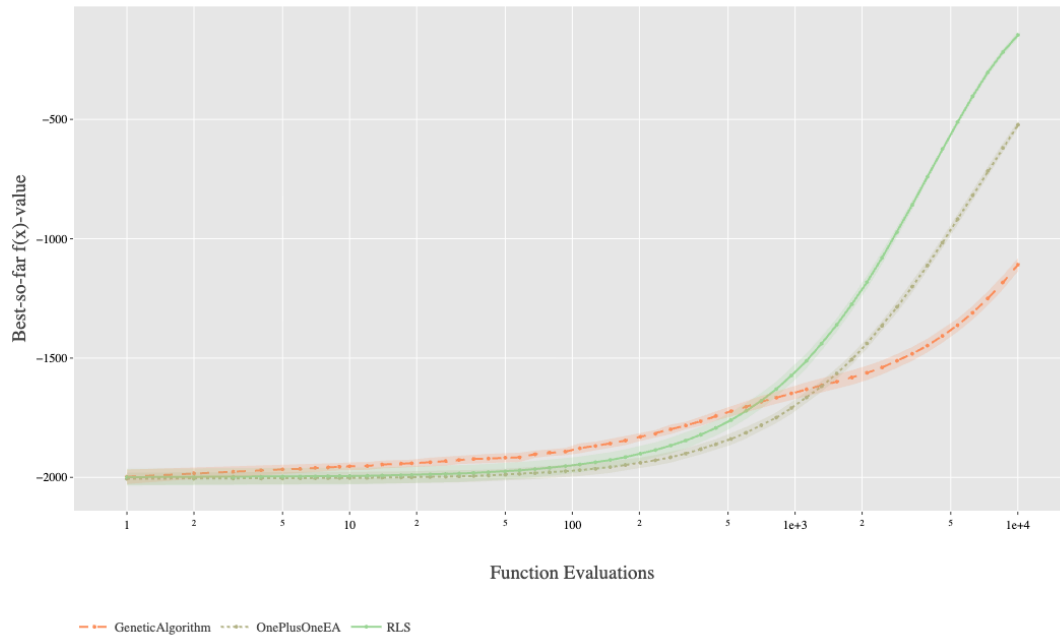
Overall for MaxCoverage: RLS is consistently the fastest to converge. OnePlusOneEA performs very similarly. My GA gets to similar final solutions but uses more evaluations. None of the algorithms seem to struggle with these problems - they all reach good solutions within the budget.

MaxInfluence Problems (2200-2203)

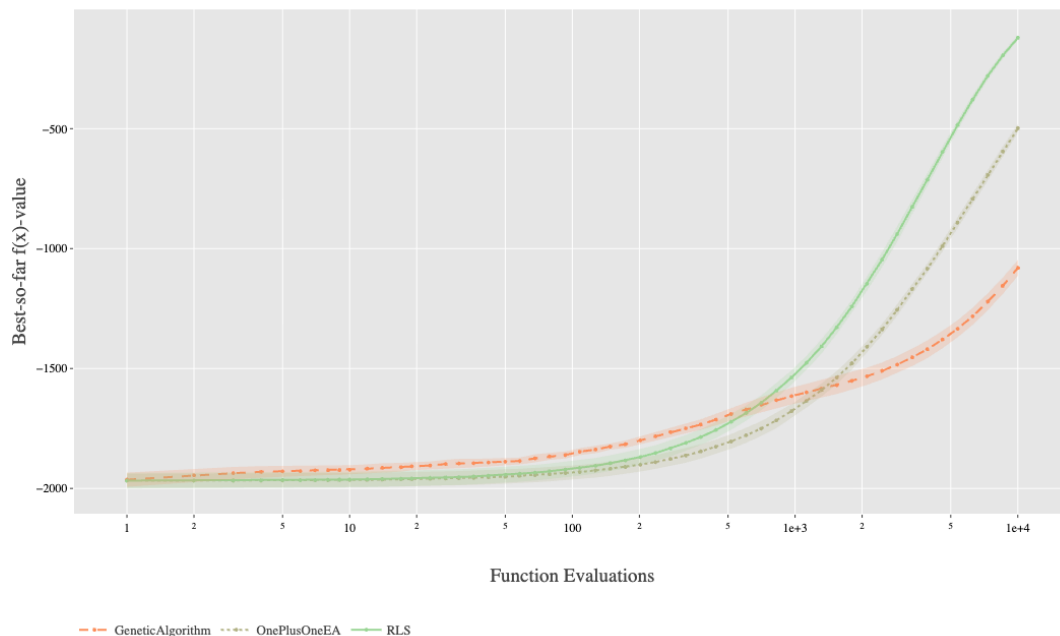
These problems look very different from MaxCoverage. The fitness values are negative (ranging from about -2000 to 0), and all three algorithms show much bigger performance differences.



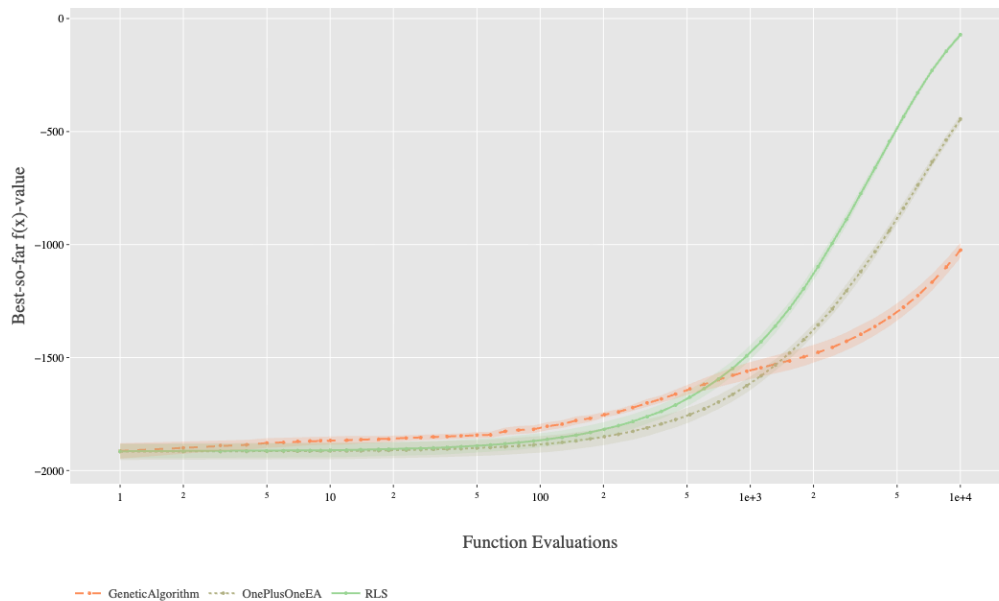
Instance 2200: RLS reaches around -200, OnePlusOneEA gets to about -500, and my GA only reaches around -1100. That's a gap of about 300 between RLS and OnePlusOneEA, and about 600 between OnePlusOneEA and the GA. None of them seem to have fully converged by 10,000 evaluations - they're all still improving.



Instance 2201: Very similar to 2200. RLS around -200, OnePlusOneEA around -500, GA around -1100. The curves look almost identical to 2200.



Instance 2202: Same pattern again. RLS $\sim$ -200, OnePlusOneEA $\sim$ -500, GA $\sim$ -1100. All three are still improving at the end of the budget.

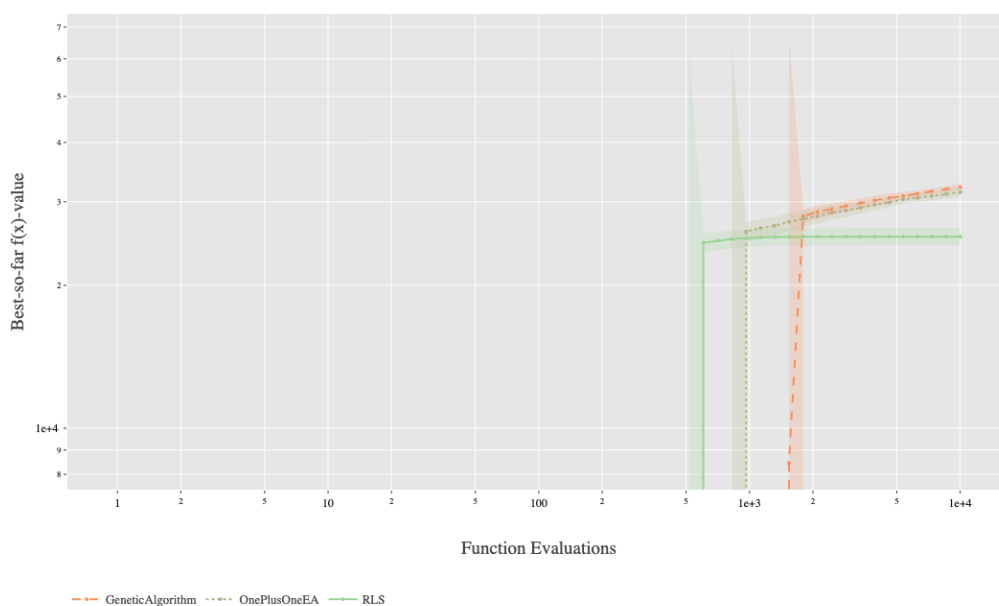


Instance 2203: Similar but slightly better fitness values. RLS reaches about -100 (better than the other MaxInfluence instances), OnePlusOneEA around -500, GA around -1050.

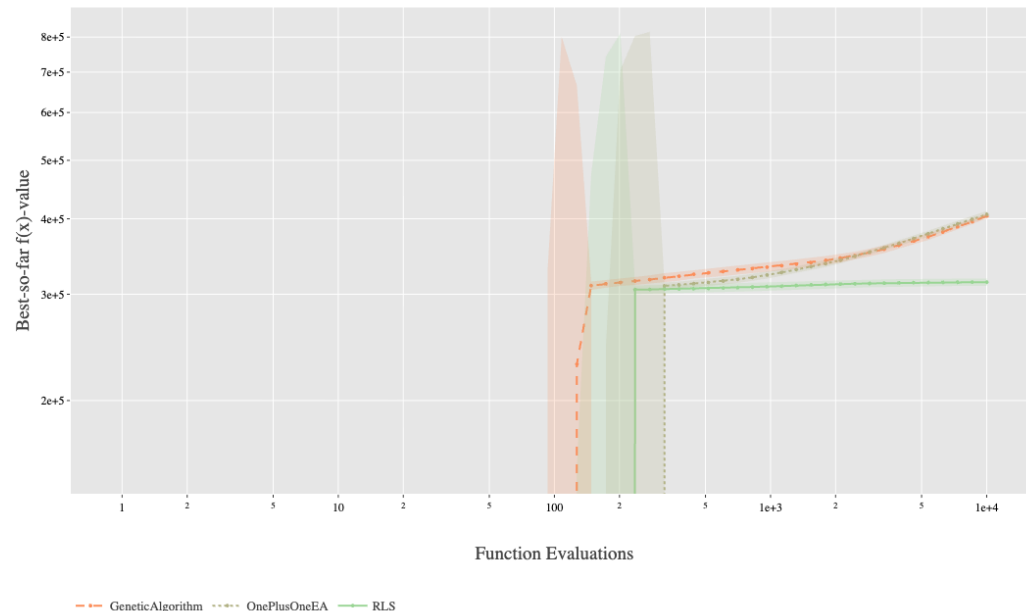
Overall for MaxInfluence: These are clearly harder problems. RLS performs significantly better than the other two algorithms - the difference is substantial (900 fitness units between RLS and GA). The negative fitness values suggest these might be minimisation problems or have some different structure. All algorithms are still improving when they hit the 10,000 evaluation limit, so a larger budget might help.

PackWhileTravel Problems (2300-2302)

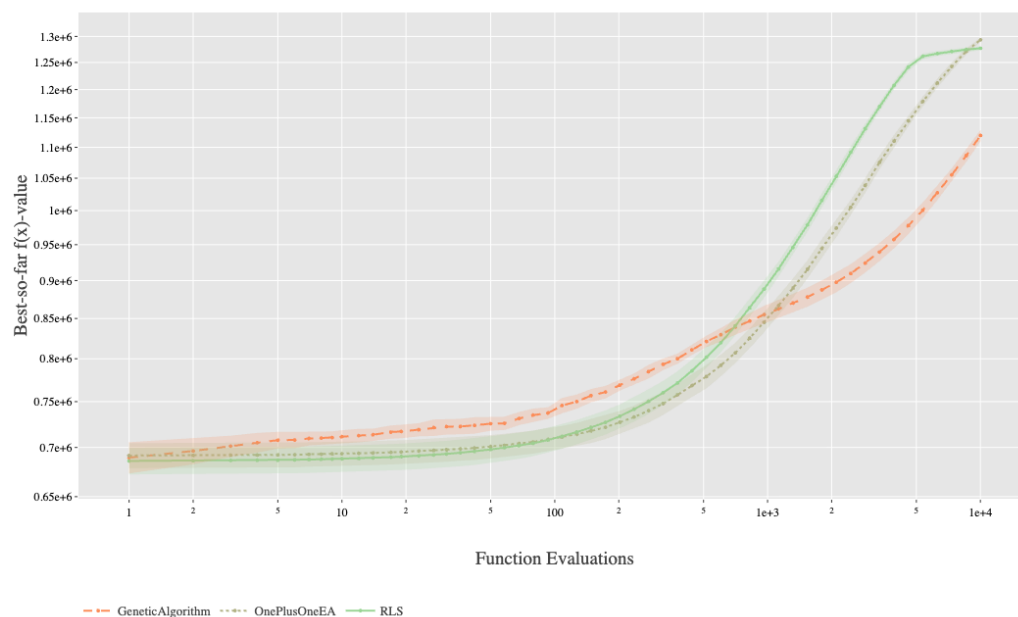
These problems are completely different from the first two types. The fitness values are huge - in the hundreds of thousands to millions - and the plots use a log scale on the y-axis.



Instance 2300: The fitness values go up to about 2.5 on the log scale (which is like $10^{2.5}$ or around 300). RLS reaches about 2.5, OnePlusOneEA shows some jumps around 1,000 evaluations and reaches similar values, and the GA starts higher (around 3×10^5 based on the scale) but grows slower. The curves have a "steppy" look to them.



Instance 2301: This one's weird. The fitness values reach 8×10^5 according to the y-axis. The GA shows a huge spike early on (around 100-200 evaluations) and reaches very high values. RLS and OnePlusOneEA grow more gradually. All three plateau pretty early, around 1,000-2,000 evaluations. The GA has really wide standard deviation bands here, meaning the runs varied a lot.



Instance 2302: Fitness values over 1×10^6 . Here the GA actually performs well - it converges fastest and reaches similar final performance to the other algorithms (all around

1.2-1.3 $\times 10^6$). RLS and OnePlusOneEA take longer to get there. This is the first instance where my GA is competitive or better.

Overall for PackWhileTravel: These problems have very different characteristics. The huge fitness scales and discrete jumps suggest they might involve integer constraints or packing decisions. Performance is much more instance-dependent here - the GA does poorly on 2300 and 2301 but well on 2302.

Algorithm Comparison

RLS

- Best overall performance across most instances (clearly best on all MaxInfluence instances)
- Fastest convergence on most MaxCoverage instances
- Tighter confidence bands (more consistent across runs)
- Struggles more on some PackWhileTravel instances

OnePlusOneEA

- Usually the second best
- Very similar to RLS on MaxCoverage problems
- Middle performance on MaxInfluence (better than GA, worse than RLS)
- Moderate performance on PackWhileTravel

Genetic Algorithm

- Slowest convergence on MaxCoverage instances but reaches a similar final fitness
- Significantly worse on MaxInfluence instances (~900 fitness units behind RLS)
- Mixed results on PackWhileTravel: poor on 2300/2301, but competitive on 2302
- Generally wider confidence bands (more variability between the runs)

Discussion

The results show that algorithm performance really depends on the problem type. RLS works best overall, especially on MaxCoverage and MaxInfluence problems. My GA doesn't perform as well on most instances, which is interesting because GAs are supposed to be good general-purpose optimisers.

The MaxInfluence problems are clearly the hardest - all algorithms are still improving at 10,000 evaluations, and the performance gaps between algorithms are largest here. It looks like these problems might benefit from a larger evaluation budget.

The PackWhileTravel problems are unusual with their massive fitness scales. The discrete improvement patterns and the fact that my GA does well on instance 2302 suggest these problems have different structure that might favor population-based approaches in some cases.

One thing I notice is that simpler algorithms (RLS, OnePlusOneEA) generally work better than my more complex GA. This matches what we learned about submodular problems that greedy-like approaches often work well for these problems.

Conclusion

RLS is the best algorithm for most of these instances. It has a faster convergence and higher quality solutions particularly on MaxCoverage problems and MaxInfluence problems. OnePlusOneEA is a secure second alternative that has the same performance though slightly lower. My GA underperforms on most instances but shows it can be competitive on certain problem structures (like PackWhileTravel 2302).

The major lesson is that the choice of algorithm is important and varies according to the type of problem. In the submodular optimisation problems that are tested in this case, simpler algorithms such as RLS appear better than more complicated population-based algorithms such as GAs.